Cloud Security Fundamentals for AI Workloads (AWS / Azure / GCP)

AI workloads add three properties classic cloud security must account for: they **hold high-value secrets** (model provider keys), they **make outbound calls to external inference APIs**, and they **process untrusted natural language that can trigger actions**. The four controls below are the load-bearing ones.

1. Secure API gateway configuration for inference endpoints

Terminate authN, rate limiting, request-size caps, and content-type pinning at a gateway *in front of* the model endpoint so the model service never sees raw, unbounded, unauthenticated traffic.

- **AWS:** API Gateway (usage plans + API keys/JWT authorizer) or ALB + **AWS WAF** (rate-based rules, body-size constraints).
- **Azure:** API Management (rate-limit/quota policies, JWT validation) + Front Door WAF.
- **GCP:** API Gateway / Apigee + Cloud Armor (rate limiting, edge security).

Caps that matter for LLMs specifically: **request body size** (a huge prompt is both a cost and a DoS vector), **per-principal request and token rate**, and **per-tier quotas**. `airte.cloud.api_gateway` encodes these as a portable policy you can also enforce as in-app middleware:

```
from airte.cloud.api_gateway import build_rate_limit_policy, validate_request
policy = build_rate_limit_policy("free")   # 20 rpm, 64KB body cap
validate_request(request, policy)
```

2. Secrets management for model API keys

Model provider keys are bearer credentials with direct billing impact — a leaked key is a denial-of-wallet and data-exfiltration risk. Rules:

- **Never** in source, container images, or plaintext env files committed to git.
- Store in **AWS Secrets Manager**, **Azure Key Vault**, or **GCP Secret Manager**.
- Grant read access to **one** secret via the workload's IAM role (no static creds); resolve at runtime; enable **automatic rotation**.
- Cache in-process for the TTL only; never log the value (redact `repr`).

`airte.cloud.secrets` models this: app code calls `get_model_key("anthropic_api_key")` against a provider interface, with concrete providers for env/in-memory (dev/test) and production sketches for **AWS Secrets Manager**, **Azure Key Vault**, and **GCP Secret Manager** (each using workload/managed identity, no static creds). The Terraform in `iac/terraform/aws-ai-gateway` provisions the rotated secret plus a least-privilege role that can read only that secret.

3. Network isolation for AI inference endpoints

- Run inference services in **private subnets** with **no public IPs**.
- **Deny egress by default**; allow-list only the provider's CIDRs/hostnames on 443. This is the network-layer twin of the SSRF allow-list in `validate_url` and the single most effective control against data exfiltration via a manipulated agent.
- Reach managed services over **private endpoints** (AWS PrivateLink / Azure Private Endpoint / GCP Private Service Connect) instead of the public internet.
- Block the **cloud metadata endpoint** (169.254.169.254) from any code path that fetches model- or user-controlled URLs (IMDSv2 / metadata-server hardening).

4. Identity-aware proxy patterns for LLM access control

Authorize access to the model per **verified identity and context**, not by network location (BeyondCorp model: AWS Verified Access, Azure AD App Proxy / Conditional Access, GCP IAP / Cloudflare Access). Decide on (user, group, model, data classification, device posture, MFA):

```
from airte.cloud.identity_proxy import IdentityAwareProxy, AccessRequest
iap = IdentityAwareProxy(model_access={"gpt-4o": frozenset({"engineering"})})
decision = iap.authorize(AccessRequest(
   user="alice", groups=frozenset({"engineering"}), model="gpt-4o",
   data_classification="confidential", mfa_present=False))
# -> ProxyDecision.STEP_UP   (confidential data requires MFA)
```

Pattern: deny if the group can't use the model or the device is non-compliant; require step-up auth for confidential data; restrict the most sensitive data to a dedicated group even with MFA.

Pulling it together

Defense in depth for an AI workload = WAF/gateway (rate + size + authN) → identity- aware authorization (who/what/which data) → private, egress-restricted runtime → vault-backed, rotated secrets → app-layer guardrails (this repo's `guardrails` and `data_pipeline`). No single layer is sufficient; the network egress allow-list and the least-agency design are the two that most often turn a successful injection into a non-event.